Plan Maestro V4.0 — Agilotext : Contexte, Output Massif, Titres, Mémoire

Version : 4.0 — Document de référence unique (Inclut le traitement des Transcripts Massifs) **Date :** 17 avril 2026 **Statut :** Proposition pour validation produit + backend
Public : Florian (produit), Nicolas (backend), devs mobile/Webflow
Changelog V4.0 : Ajout de la Vague 9 (Architecture Output Map-Reduce pour les réunions de 3h+ et PV CSE limités par les tokens).

Objectif : Viser le **meilleur niveau marché** pour professions exigeantes (notaires, avocats, pharmaciens) et grand public, **sans casser** l'existant — en particulier les **templates HTML**, les **modèles avec questions**, le **Word Boost compte**, et le flux `upload → transcript → CR`.

Documents de référence croisés

Plans & Cahiers des charges internes :

- PLAN_MAESTRO_V2 — Plan consolidé Speechmatics + Speaker ID
- PLAN_GLOBAL_V2 — Refonte contexte, profil, titres, speaker
- CDC Contexte — Cahier des charges contexte amont (2nd dev)
- CDC Titres — Spéc. titres intelligents & organisation
- Plan 20/20 Titres — Exécution titres & dossiers
- Backend Extensions Spec — Endpoints setJobContext, addJobContextAttachment
- Brief Nicolas Titres — Brief technique pour Nicolas
- Brief Backend Statut CR — Statut CR & redo

Code Frontend — Scripts Webflow (CDN GitHub) :

- ent.js — Dashboard Business (upload, FilePond, sendMultipleAudio)
- pro.js — Dashboard Pro
- free.js — Dashboard Free
- mount-streaming.js — Streaming live (AudioWorklet → WAV)
- Code-main-editor.js — Éditeur principal transcript
- Code-modeles-compte-rendu.js — Chips modèles CR (getPromptModelsUserInfo/StandardInfo)
- relance-compte-rendu.js — Relance CR (redoSummary + promptId + compteur)
- Code-changement-audio.js — Rail de changement de job (liste, tri, recherche)
- Code-chat.js — Chat IA
- Code-save_transcript.js — Auto-save transcript
- orchestrator.js — Orchestration jobs + credentials
- token-resolver.js — Résolution tokens
- FEATURES_TRACKING.md — Mapping Webflow Embed ↔ GitHub

Code Mobile :

- api.ts — uploadAudio, getJobsInfo, mapping DTO
- JobCard.tsx — Composant job (title, filename)
- JobsListScreen.tsx — Liste jobs + renommage

Backend Java (références connues) :

- `FutureSetWordBoost2.java` — Logique Word Boost
- `WordBoostDAO2` / `PostWordBoostTreater` — Pipeline boost → Speechmatics additional_vocab

- `GptNewSummaryMaker` / `GptBaseNewChunkOneExecutor` — Génération CR (prompt + chunking)
- `SpeechmaticsAiTranscriptor` → `sendAudioHolder` — Envoi ASR
- Tables SQL : `word_boost_2`, `word_boost_name_2`, `job_word_boost_2`, `user_default_word_boost_2`

Documentation externe :

- Speechmatics Speaker Identification
- Speechmatics Batch Speaker ID
- Graphify — Knowledge Graph — Inspiration V8 Mémoire de Dossier
- Transcript Graphify — Analyse vidéo Graphify

---

1. Fonctionnement Actuel — Audit Complet

1.1 Pipeline audio → transcript → CR

```
flowchart LR
    subgraph "Frontend"
        WEB["Dashboard Webflow<br>(ent.js / pro.js / free.js)"]
        MOB["App Mobile<br>(api.ts → uploadAudio)"]
        LIVE["Streaming Live<br>(agilo-live-transcribe.js)"]
    end

    subgraph "Backend Java"
        SEND["sendMultipleAudio<br>sendYoutubeUrl"]
        SM["Speechmatics ASR<br>+ Word Boost compte"]
        POST["Post-traitement<br>(format, dictionnaires)"]
        CR["GptNewSummaryMaker<br>prompt + template + transcript"]
    end

    WEB --> SEND
    MOB --> SEND
    LIVE -->|"WAV final"| SEND
    SEND --> SM
    SM --> POST
    POST -->|"si doSummary"| CR
```

1.2 Système de Templates / Modèles de CR (CRITIQUE — ne pas casser)

Le système actuel fonctionne ainsi :

```
┌──────────────────────────────────────────────────────────────┐
│  MODÈLES DE COMPTE-RENDU — FONCTIONNEMENT ACTUEL             │
│                                                              │
│  1. TYPES DE MODÈLES                                         │
│     → Standards (promptModelId < 100) : créés par Agilotext  │
│     → Personnalisés (promptModelId ≥ 100) : créés par le     │
│       client via l'éditeur de modèles (questions + HTML)     │
│                                                              │
│  2. APIs EXISTANTES                                          │
│     → getPromptModelsUserInfo → liste modèles utilisateur    │
```

```
│     → getPromptModelsStandardInfo → liste modèles standards  │
│     → createPromptModelUser → création (questions résolues)  │
│     → getPromptModelUserStatus → polling statut création     │
│     → setJobPromptModel → associer un promptId à un job      │
│                                                              │
│  3. CE QUE CONTIENT UN MODÈLE CLIENT                         │
│     → Prompt long avec HTML structuré (titres, tableaux,     │
│       sections imposées, variables issues des questions)     │
│     → Les "questions" sont posées à la création et leurs     │
│       réponses sont injectées dans le prompt comme variables │
│     → Le HTML imposé = RÉFÉRENCE DE FORME du livrable       │
│                                                              │
│  4. UTILISATION                                              │
│     → Premier CR : via sendMultipleAudio avec promptId       │
│     → Relance : via redoSummary avec autre promptId          │
│     → Code-modeles-compte-rendu.js affiche les chips         │
│     → relance-compte-rendu.js gère le redo + countdown      │
│                                                              │
│  5. FLUX GptNewSummaryMaker ACTUEL                           │
│     → Entrée : transcript (string)                           │
│     → Prompt : template utilisateur (HTML + variables) +     │
│       phrase "Here is the transcript to summarize"           │
│     → GptBaseNewChunkOneExecutor gère le budget tokens       │
│     → AUCUN bloc "contexte document" injecté aujourd'hui     │
└──────────────────────────────────────────────────────────────┘
```

> *[!CAUTION]* ***Actifs à protéger absolument :***
>
> - *Templates **HTML** avec sections, tableaux, structures imposées*
> - *Variables issues des **questions** résolues à la création du modèle*
> - *Le fonctionnement de `redoSummary` avec changement de `promptId`*
> - *Le Word Boost **permanent** des utilisateurs*
> - *La rétrocompatibilité API (mobile sans nouveaux champs = inchangé)*

1.3 APIs et Endpoints Existants

Endpoint	Méthode	Rôle	Prod ?
`sendMultipleAudio`	POST multipart	Upload audio + `promptId`	✅
`sendYoutubeUrl`	POST form	YouTube	✅
`getTranscriptStatus`	GET	Polling	✅
`receiveText` / `receiveSummary`	GET	Résultat	✅
`redoSummary`	GET	Relance CR	✅
`getPromptModelsUserInfo`	GET	Modèles user	✅
`getPromptModelsStandardInfo`	GET	Modèles standards	✅
`createPromptModelUser`	POST	Créer modèle	✅

`setJobPromptModel`	POST	Associer prompt→job	✅
`setWordBoostList2` / `getWordBoost2`	POST/GET	Word Boost compte	✅
`renameTranscriptFile`	POST	Renommer fichier	✅
`setJobContext`	POST	Contexte par job	⚠️ Spécifié, non utilisé
`addJobContextAttachment`	POST multipart	PJ par job	⚠️ Spécifié, non utilisé
`getJobContext`	GET	Lecture contexte	⚠️ Spécifié, non utilisé

1.4 Données Memberstack / Onboarding disponibles

Source	Données	Fiabilité
Memberstack (auth)	Prénom, nom, email, plan (free/pro/ent)	✅ Fiable — identité du compte
Onboarding	Outil visio (Teams/Meet/Zoom/tel), fréquence réunions, persona (étudiant/salarié/dirigeant...), cas d'usage (réunion équipe/client/recrutement...)	🟡 Déclaratif — indice, pas vérité
Futur	Empreintes vocales, préférences, équipe/contacts	🔮 À développer

---

2. Concept Unifié : `JobContext` — Sources Typées

L'erreur serait de créer N pipelines parallèles pour chaque source de contexte. **Une seule abstraction** avec des sources typées :

```
flowchart TB
    subgraph "Sources de contexte"
        DOC["📄 Document joint<br>(PDF/DOCX/TXT)"]
        PROFILE["👤 Profil utilisateur<br>(Memberstack + onboarding)"]
        USERFIELDS["✏️ Champs UI<br>(date réunion, note...)"]
        MCP["🔌 MCP / Mail<br>(futur V7)"]
    end

    subgraph "JobContext (par jobId)"
        ANALYZE["Analyse LLM<br>(JSON structuré)"]
        META["Métadonnées<br>(date, lieu, type)"]
        BOOST["Word Boost<br>éphémère"]
        PROMPT_CTX["Bloc texte<br>pour prompt CR"]
    end

    DOC -->|"extraction + LLM"| ANALYZE
    PROFILE -->|"snapshot léger"| PROMPT_CTX
```

```
    USERFIELDS -->|"confirmé par user"| META
    MCP -->|"même pipeline"| ANALYZE

    ANALYZE --> BOOST
    ANALYZE --> META
    ANALYZE --> PROMPT_CTX
```

Règle de priorité factuelle

```
Document joint  >  Champs UI confirmés  >  Profil utilisateur/onboarding
     (ODJ)         (date, participants)       (persona, outil visio)
```

Un persona "dirigeant" ne remplace **jamais** une date extraite d'un ODJ. Le profil sert de **contexte faible** (ton, vocabulaire attendu), pas de source de faits.

---

3. Le Nouveau Flux COMPLET ne Casse Pas les Templates

3.1 Ordonnancement strict des blocs dans le prompt CR

```
┌──────────────────────────────────────────────────────────┐
│  PROMPT FINAL (envoyé à GptNewSummaryMaker)               │
│                                                           │
│  BLOC 1 — Instructions système Agilotext (fixe)           │
│           Rôle, sécurité, langue                          │
│           (~200 tokens)                                   │
│                                                           │
│  BLOC 2 — MODÈLE UTILISATEUR (INCHANGÉ ✅)                 │
│           = Le prompt du template (HTML + questions)       │
│           = Référence de FORME du livrable                 │
│           (~500-3000 tokens selon modèle)                  │
│                                                           │
│  BLOC 3 — CONTEXTE FACTUEL (NOUVEAU 🆕)                    │
│           = Texte plat, résumé, plafonné                   │
│           = NE contient PAS de HTML qui mime le template    │
│           = NE remplace PAS les variables du modèle         │
│           Max : 2000 tokens                                │
│           Contenu type :                                   │
│           « INFORMATIONS CONTEXTUELLES FOURNIES :           │
│             Date réunion : 12 avril 2026                   │
│             Participants : Jean Dupont (DG), ...           │
│             Ordre du jour :                                │
│               1. Bilan Q1                                  │
│               2. Recrutement                               │
│             Rédacteur/Compte : Marie Martin (Pro)          │
│             Acronymes : CODIR = Comité de Direction »      │
│                                                           │
│  BLOC 4 — TRANSCRIPT                                      │
│           = la transcription audio (chunking existant)      │
│           = budget = fenêtre - Bloc1 - Bloc2 - Bloc3       │
│                                                           │
```

```
│  RÈGLE DE COMPRESSION :                                    │
│  Si budget insuffisant :                                   │
│    1. Réduire Bloc 3 (pas Bloc 2 !)                        │
│    2. Minimum Bloc 3 = participants + date seulement       │
│    3. Ne JAMAIS toucher Bloc 2 (= contrat client)          │
└────────────────────────────────────────────────────────────┘
```

> *[!IMPORTANT]* ***Ce qu'il ne faut SURTOUT PAS faire :***
>
> - *Injecter du JSON brut dans le template HTML*
> - *Pré-pendre un pavé énorme **avant** le template (pousse les consignes hors fenêtre)*
> - *Laisser le LLM **réécrire** le HTML du modèle*
> - *Fusionner le document joint avec les variables-questions du template*
> - *Écraser le `promptId` du client par un "prompt enrichi" synthétique*

3.2 `redoSummary` — Cycle de vie du contexte

Scénario	Comportement
1er CR	Contexte document injecté en Bloc 3
`redoSummary` **même** template	✅ Même Bloc 3, idempotent
`redoSummary` **autre** template	✅ Bloc 2 change (nouveau promptId), Bloc 3 reste identique
Utilisateur corrige la date	✅ Bloc 3 mis à jour, flag `contextVersion` incrémenté
Utilisateur supprime le document	✅ Bloc 3 vidé, redo sans contexte
Audit/traçabilité (notaires)	Stockage `contextVersion` + hash → permet de prouver quel contexte a produit quel CR

---

4. Pipeline Technique — Architecture Détaillée

4.1 Cas 1 : Upload fichier + document contexte (chemin « tout en un POST »)

```
sequenceDiagram
    participant U as Utilisateur
    participant FE as Dashboard (ent.js)
    participant API as Backend Java
    participant TIKA as Extraction (Tika/POI)
    participant LLM_A as LLM Analyse (Haiku)
    participant SM as Speechmatics
    participant LLM_CR as LLM CR (GPT-4)

    U->>FE: Audio + Document + Options
    FE->>API: POST sendMultipleAudio<br>(fileUpload1 + contextFile + promptId)

    Note over API: ⏱️ Phase séquentielle (2-4s)
    API->>TIKA: Extraction texte du document
    TIKA-->>API: Texte brut
```

```
    API->>LLM_A: Analyse rapide (participants, dates, termes)
    LLM_A-->>API: JSON structuré

    Note over API: Construction Word Boost éphémère
    API->>API: Merge boost permanent + boost document<br>Plafond 300 termes,
dédoublonnage

    API->>SM: Audio + additional_vocab enrichi
    SM-->>API: Transcript brut

    API->>API: Post-traitement (format, dictionnaires)

    opt doSummary = true
        API->>API: Construction prompt<br>Bloc1 + Bloc2(template) + Bloc3(contexte)
+ Bloc4(transcript)
        API->>LLM_CR: Prompt enrichi
        LLM_CR-->>API: CR HTML
    end

    API-->>FE: jobId (immédiat)
    FE->>API: Poll getTranscriptStatus
```

> *[!WARNING]* ***Honnêteté latence :*** *L'analyse du document (Tika + LLM = 2-4s) est* ***séquentielle*** *avant l'envoi à Speechmatics. Pour un audio de 2 min (traitement rapide), c'est un surcoût de +20%. Pour un audio d'1 heure, c'est <1%. Le cas* `preAnalyze + contextId` *est le seul vrai "zéro impact".*

4.2 Cas 2 : Enregistrement Live + document pré-analysé (zéro latence ajoutée)

```
sequenceDiagram
    participant U as Utilisateur
    participant FE as Dashboard
    participant API as Backend Java

    U->>FE: Glisse le document de contexte
    FE->>API: POST preAnalyzeContextDocument
    API-->>FE: contextId + preview (noms, termes)

    Note over U,FE: L'utilisateur voit "5 noms détectés"
    Note over U,FE: L'utilisateur démarre l'enregistrement
    Note over U,FE: ... enregistrement en cours ...

    U->>FE: Stop → WAV construit
    FE->>API: POST sendMultipleAudio<br>(fileUpload1 + contextId)

    Note over API: Pas d'analyse doc = déjà fait !
    API->>API: Récupère le contexte par contextId
    API->>API: Word Boost = déjà prêt
    Note over API: Envoi immédiat à Speechmatics
```

4.3 Endpoints

Extension `sendMultipleAudio` (rétrocompatible)

```
POST /api/v1/sendMultipleAudio
  Content-Type: multipart/form-data

  Champs existants (INCHANGÉS) :
  - fileUpload1, username, token, edition, doSummary,
    timestampTranscript, speakersExpected, formatTranscript,
    translateTo, deviceId, mailTranscription, promptId, title

  NOUVEAUX (tous optionnels — absent = comportement identique) :
  - contextFile     (file; pdf/docx/doc/txt/rtf, max offre)
  - contextId       (string; ref. preAnalyze, mutuellement exclusif avec contextFile)
  - meetingDate     (string ISO; date réunion explicite, modifiable par user)
  - meetingSource   (string; "live_recorder" | "file_upload" | "youtube")
```

Pré-analyse (NOUVEAU)

```
POST /api/v1/preAnalyzeContextDocument
  Content-Type: multipart/form-data
  - username, token, edition (requis)
  - contextFile (file, requis)

Réponse succès :
{
  "status": "OK",
  "contextId": "ctx_abc123",
  "analysis": {
    "date_reunion": "2026-04-12",
    "date_confiance": 0.95,
    "participants": [
      {"prenom": "Jean", "nom": "Dupont", "role": "DG", "confiance": 0.98}
    ],
    "sujets": ["Bilan Q1", "Recrutement"],
    "acronymes": {"CODIR": "Comité de Direction"},
    "termes_techniques": ["EBITDA", "NPS"],
    "wordBoostPreview": ["Jean Dupont", "CODIR", "EBITDA"]
  },
  "warnings": ["8 termes ajoutés au vocabulaire"]
}

TTL contextId : 2 heures. Nettoyage cron.

Erreur (PDF image) :
{
  "status": "KO",
  "errorMessage": "error_document_no_text_content",
  "userMessage": "Document scanné détecté. L'analyse n'est pas possible."
}
```

4.4 Word Boost Éphémère — Règles Complètes

```
CONSTRUCTION :
  1. Charger boost permanent de l'utilisateur
  2. Extraire termes du document (confiance ≥ 0.70 pour les noms)
  3. Dédoublonnage case-insensitive + normalisation accents
  4. Priorité si > plafond :
     1er : NOMS PROPRES (valeur #1 pour l'utilisateur)
     2ème : Acronymes du document
     3ème : Termes techniques
     4ème : Boost permanent existant (déjà connu par Speechmatics)

PLAFONDS PAR OFFRE :
  Free    → 50 termes max
  Pro     → 200 termes max
  Business → 300 termes max
  (marge technique Speechmatics ≈ 1000)

CYCLE DE VIE :
  → Créé au lancement du job
  → Envoyé dans additional_vocab de Speechmatics
  → Supprimé/désassocié après READY ou ON_ERROR
  → JAMAIS écrit dans le boost permanent du compte
  → Tables existantes job_word_boost_2 adaptables (à valider Nicolas)
```

4.5 Protection contre les Hallucinations LLM

```
COUCHE 1 — PROMPT DÉFENSIF
  "Extrais UNIQUEMENT les informations EXPLICITEMENT présentes.
   Ne déduis ni n'invente rien. Attribue un score de confiance
   0.0-1.0 à chaque participant détecté."

COUCHE 2 — FILTRE POST-LLM (serveur Java)
  → Rejeter participants avec confiance < 0.70
  → Rejeter termes < 2 caractères
  → Rejeter stopwords français courants
  → Plafond dur : 50 noms max, 100 termes techniques max

COUCHE 3 — PREVIEW UTILISATEUR (recommandé dès Vague 1)
  → "5 participants et 8 termes détectés dans votre document"
  → L'utilisateur PEUT retirer des faux positifs
  → Pour professions réglementées : ce n'est PAS du polish
```

---

5. Profil Utilisateur — `UserProfileContext`

5.1 Sources et usage

Source	Données	Usage dans le prompt (Bloc 3)

Memberstack	Prénom, nom, email, plan	Ligne "Rédacteur/Compte : ..." — pas liste de participants tiers
Onboarding	Outil visio, fréquence, persona, cas d'usage	**Indicatif** pour ton/vocabulaire ("contexte organisationnel supposé")
Préférences (futur)	Langue, dossier par défaut, format préféré	Paramètres UX

5.2 Règles

- **Budget tokens** profil dans le prompt : 300-800 tokens max
- **Jamais** source de faits sur la réunion (pas "la réunion est dirigée par Florian Bauer" juste parce qu'il est le titulaire du compte)
- **Snapshot** optionnel par job pour audit
- **Variables onboarding versionnées** (schéma) → ne pas casser les anciens jobs quand on ajoute des champs

5.3 Injection possible (dès V5, préparé dès V1)

```
Bloc 3 (section "Contexte utilisateur") :
  « Le rédacteur utilise habituellement Teams pour ses réunions.
    Profil : Dirigeant / Fondateur.
    Cas d'usage typique : Réunion d'équipe, rendez-vous client. »
```

Ce texte est **faible** — il ne prime **jamais** sur le document joint ni sur les champs UI confirmés.

---

6. Speaker Identification — Conforme à la Doc Speechmatics

6.1 Ce que la doc officielle prescrit

> *[!IMPORTANT] Basé sur les pages Speechmatics consultées :*
>
> - *Speaker identification*
> - *Batch speaker identification*

Flux officiel :

```
ENROLLMENT (inscription)
  → Extraits audio 5-30s, locuteur SEUL
  → Pour robustesse : PLUSIEURS clips par personne sous
    différentes conditions acoustiques (laptop vs dictaphone)
  → Résultat : speaker_identifiers (chaînes opaques chiffrées)

IDENTIFICATION (utilisation)
  → Passer les identifiers dans speaker_diarization_config :
    {
      "speakers": [
        {"label": "Me Dupont", "speaker_identifiers": ["<id1>", "<id2>"]},
        {"label": "Alice",     "speaker_identifiers": ["<id3>"]}
      ]
    }
```

```
  → Le moteur étiquette les segments avec les labels
  → speakers_sensitivity ajuste le matching
```

Limites doc :

- **50 identifiants max** par session
- **Minimiser** le nombre d'IDs pour **meilleure précision**
- Labels réservés interdits (`S1` , `S2` , `UU`)
- IDs **liés au modèle/operating point** → **régénération** si le modèle change
- IDs **chiffrés, par client, par projet** — pas de partage cross-compte

6.2 Critique de la fusion WAV

Approche	Avis
Enrollment → `speaker_identifiers` → config JSON	✅ **Conforme** doc officielle
Concaténer WAV référence + WAV réunion en un seul fichier	❌ **Hors doc**, risque VAD, décalage temporel, qualité dégradée
Un seul clip d'enrollment "universel" tous micros	⚠️ **Insuffisant** — la doc recommande PLUSIEURS clips sous conditions différentes

6.3 Produit Agilotext — Vision

```
FONCTIONNEMENT CIBLE :
  1. Utilisateur enregistre sa propre voix (5-15s) → enrollment
  2. Invite des collègues via lien (5-15s chacun) → enrollment
  3. Les speaker_identifiers sont stockés côté Agilotext
     (mapping: identifier → nom + userId/contact)
  4. À chaque transcription : les identifiers sont passés dans
     speaker_diarization_config → étiquetage automatique
  5. Quotas par offre (ex: 5 voix Free, 10 Pro, 30 Business)

HONNÊTETÉ PRODUIT :
  → Améliore l'étiquetage, pas de garantie parfaite
  → Micro différent = ré-enrollment recommandé (ou multi-clips)
  → Changement modèle Speechmatics = régénération des IDs
```

6.4 Word Boost ≠ Speaker ID

Les deux sont **orthogonaux et complémentaires** :

- **Word Boost** (`additional_vocab`) = meilleure **reconnaissance des mots** (noms, termes)
- **Speaker ID** (`speaker_diarization_config`) = meilleur **étiquetage des locuteurs**

Pas besoin de fusionner des fichiers audio pour combiner les deux.

---

7. Titres Intelligents & Dossiers

7.1 Architecture

```
CONCEPTS À SÉPARER :
  originalUploadFileName  → Immutable, fichier reçu à l'upload
  displayTitle            → Titre affiché (user, IA, ou default)
  filename / clé stockage → Clé technique stable pour liens/exports
  titleSource             → "user" | "ai" | "default" | "filename"

RÈGLES :
  → displayTitle = PRIORITÉ 1 pour l'affichage
  → Ne JAMAIS écraser un displayTitle posé par l'utilisateur
  → Titre IA = suggestion SEULEMENT si displayTitle est vide/auto
  → Le nom du fichier uploadé reste accessible (fiche détail)
```

7.2 Titre IA — Source de données

Le titre IA se génère **après** le transcript, à partir de :

1. **Extrait transcript** (plafonné, premiers segments)
2. **Résumé** `JobContext` (sujets ODJ si document joint)
3. **Indices onboarding** (poids faible — persona, type d'usage)

→ Pas besoin d'un nouveau modèle : un appel LLM léger (Haiku/4o-mini) sur un extrait court, avec prompt "Génère un titre concis de 5-10 mots max".

7.3 Ce que ça change dans le code

Composant	Modification
Backend `getJobsInfo`	Exposer `displayTitle`, `titleSource`, `folderId`
Backend `PATCH /jobs/{jobId}`	Nouveau endpoint pour title + folder
Mobile `api.ts`	Mapper `displayTitle` + helper `getJobDisplayTitle()`
Mobile `JobsListScreen.tsx`	Renommage → PATCH au lieu de `renameTranscriptFile`
Webflow `Code-changement-audio.js`	Utiliser `displayTitle` pour l'affichage rail

7.4 Dossiers (MVP)

- Plat (pas de sous-dossiers V1)
- `folderId` nullable sur le job
- CRUD `/folders` + `PATCH /jobs/{id}` avec `folderId`
- Soft delete dossier → jobs reviennent à la racine
- `jobCount` dans GET /folders pour éviter N+1

---

8. Auto-Nommage Post-Transcription

Le plan est de **générer automatiquement** un titre lisible dès que le transcript est prêt. Ce titre devient le `displayTitle` si personne n'a nommé le fichier manuellement.

Flux :

```
1. Job créé → displayTitle = null, titleSource = "default"
2. Transcript prêt → tâche async : LLM titre sur extrait
```

```
3. Titre IA généré → displayTitle = "Comité de direction Q1",
   titleSource = "ai" (SEULEMENT si pas déjà user-set)
4. Utilisateur renomme → displayTitle = "Mon titre",
   titleSource = "user" (JAMAIS écrasé par l'IA ensuite)
```

---

9. MCP Agilotext & Messagerie — Phase Ultérieure

- Le MCP est une **source typée** comme les autres dans `JobContext`
- Même pipeline sécurité/TTL/quotas
- **Hors** chemin critique V1-V5
- À brancher quand l'abstraction `JobContext` est stable

---

10. Sécurité & RGPD

```
UPLOAD DOCUMENT :
  ✓ Taille max par offre (Free 2MB, Pro 5MB, Ent 10MB)
  ✓ MIME whitelist (magic bytes, pas juste extension)
    → pdf, docx, doc, txt, rtf
    ✗ PAS html, js, svg, zip, exe
  ✓ PDF macros/JS embarqué → rejeter
  ✓ Stockage isolé par job, chiffrement au repos
  ✓ Nom fichier sanitisé (pas injection path)

RGPD :
  ✓ TTL aligné sur rétention audio du compte
  ✓ Option : supprimer le document source après extraction
    (ne garder que le JSON d'analyse)
  ✓ Droit à l'oubli : suppression job = suppression doc
  ✓ Speaker IDs : consentement explicite + TTL + purge

PDF SCANNÉS :
  → Tika retourne "" → message user clair
  → OCR serveur (Tesseract/Vision) = hors V1
```

---

11. Quotas par Offre

Offre	Doc max	Analyses/mois	Boost éphémère	Speaker IDs	Dossiers
Free	2 MB	10	50 termes	5 voix	3
Pro	5 MB	100	200 termes	10 voix	20
Business	10 MB	Illimité	300 termes	30 voix	Illimité

---

12. Feuille de Route par Vagues

```
gantt
    title Feuille de route Agilotext V2 — 8 vagues
```

```
    dateFormat  YYYY-MM-DD
    section Vague 1 - Contexte
    Validation upload + extraction Tika     :v1a, 2026-05-01, 3d
    Analyse LLM + Word Boost éphémère       :v1b, after v1a, 4d
    preAnalyze + contextId (live)           :v1c, after v1b, 2d
    Frontend zone PJ + preview             :v1d, after v1a, 3d
    section Vague 2 - CR Enrichi
    Injection Bloc 3 dans GptNewSummaryMaker :v2a, after v1c, 3d
    Budget tokens global + redo             :v2b, after v2a, 2d
    Dates confirmables UI                   :v2c, after v2a, 2d
    section Vague 3 - Titres
    DTO displayTitle + PATCH job            :v3a, after v2b, 3d
    Titre IA async                          :v3b, after v3a, 2d
    Frontend + mobile mapping               :v3c, after v3b, 3d
    section Vague 4 - Dossiers
    CRUD /folders                           :v4a, after v3c, 3d
    UI web + mobile                         :v4b, after v4a, 3d
    section Vague 5 - Profil
    UserProfileContext sync                 :v5a, after v4b, 3d
    Injection profil Bloc 3                 :v5b, after v5a, 2d
    section Vague 6 - Speaker ID
    Enrollment + invitations                :v6a, after v5b, 5d
    Intégration Speechmatics ID             :v6b, after v6a, 5d
    section Vague 7 - MCP
    MCP vers JobContext                     :v7a, after v6b, 5d
    section Vague 8 - Mémoire Dossier
    Graphe cumulatif par dossier            :v8a, after v7a, 5d
    Word Boost persistant dossier           :v8b, after v8a, 3d
    CR avec mémoire réunions précédentes    :v8c, after v8b, 4d
```

Résumé des vagues

Vague	Livrables	Durée est.	Dépendances
V1	Upload doc, extraction, analyse LLM, Word Boost éphémère, preAnalyze, preview	~9 jours	Aucune
V2	Injection Bloc 3 dans CR, dates confirmables, redo stable	~7 jours	V1
V3	displayTitle, titre IA, PATCH job, mapping mobile	~8 jours	Backend V2
V4	Dossiers CRUD + UI	~6 jours	V3
V5	Profil utilisateur enrichi (Memberstack + onboarding)	~5 jours	V2
V6	Speaker Identification Speechmatics (enrollment, invitations, quotas)	~10 jours	R&D Nicolas
V7	MCP / mail → JobContext	~5 jours	V2 stable
V8	**Mémoire de Dossier** — graphe cumulatif, boost persistant, CR cross-réunion	~12 jours	V4 + V2

---

13. Stratégie de Déploiement & Structure du Backlog

13.1 Stratégie "Zéro Casse" (Rétrocompatibilité)

Le plus grand risque de ce projet serait de lancer une "API V2" en repartant de zéro ou de refaire simultanément tout le frontend. Le plan Maestro V3 est sécurisé par cette stratégie :

1. **Pas de Backend V2 :** L'endpoint central `sendMultipleAudio` ne change pas de contrat de base. Il accepte simplement de **nouveaux champs optionnels** (`contextFile` , `folderId`). Si l'application Mobile actuelle envoie une requête sans ces champs, l'API se comporte exactement comme aujourd'hui.
2. **Pas de refonte Web App (pour l'instant) :** Refaire le frontend en React/Vue.js prendrait des mois. Nous poussons ces nouvelles fonctionnalités sur le **Webflow actuel** en modifiant finement les scripts JS existants (voir Annexe B). L'objectif est le Time-to-Market (2-3 mois).
3. **Refonte reportée à 2027 :** Seulement quand l'intelligence serveur (Contexte, Dossiers, Mémoire V8) sera live et génératrice de revenus, la refonte du Webflow vers une App React/Vue sera justifiée.

13.2 Structure du Backlog (Epics & Tickets)

Pour organiser le travail (Jira, Linear, Trello), l'**Epic Globale** s'appellera : **Agilotext V2.0 — Le Moteur Contextuel**. Elle se divise en sous-Epics correspondant à nos Vagues :

- **Epic 1 : [Moteur Contextuel] – Extraction Doc & Word Boost Éphémère (V1)**
  - *Sous-tâches : Intégration Tika, LLM Analyse, Modif `sendMultipleAudio` , Dropzone frontend (ent.js).*
- **Epic 2 : [Moteur Contextuel] – Injection Bloc 3 & Sécurisation Templates (V2)**
  - *Sous-tâches : Prompting GptNewSummaryMaker, Tests non-régression Redo.*
- **Epic 3 : [UX & Organisation] – Titres Intelligents (V3)**
  - *Sous-tâches : API `displayTitle` , Tâche IA asynchrone, intégration Mobile.*
- **Epic 4 : [UX & Organisation] – CRUD Dossiers & Affichage (V4)**
  - *Sous-tâches : Modèles base de données Dossiers, UI Webflow/Mobile.*
- **Epic 5 : [Moteur Contextuel] – Métadonnées Profil & Onboarding (V5)**
  - *Sous-tâches : Snapshot Memberstack, injection légère dans Bloc 3.*
- **Epic 6 : [Précision Audio] – Speaker Identification Native (V6)**
  - *Sous-tâches : Enrollment officiel Speechmatics, DB `speaker_identifiers` , Quotas.*
- **Epic 7 : [Intégration] – Support Agilotext MCP (V7)**
  - *Sous-tâches : Mapping de la source MCP vers `JobContext` .*
- **Epic 8 : [Moat / Innovation] – Mémoire Persistante de Dossier (V8)**
  - *Sous-tâches : JSON cumulatif (FolderContext), Word Boost de dossier, Hook de résumé post-réunion.*

Règle pour les développeurs : « Toute documentation technique se trouve dans le document Plan Maestro (ce fichier). Il fait foi en cas de doute. »

---

14. Vague 9 — Architecture Output "Grand Format" (PV CSE, 3h+)

> *[!IMPORTANT] **Le défi :** Pour les réunions de 3 à 4 heures (CSE, Conseils d'Administration), la transcription génère entre 30 000 et 50 000 mots. L'envoi direct de cette masse dans un seul prompt Mistral (ou Claude/OpenAI) provoque deux problèmes majeurs :*

> 1. ***Dépassement de la fenêtre de contexte*** *(Rate limit, erreurs API, coûts explosifs).*
> 2. ***"Lost in the middle"*** *(Le modèle oublie des détails cruciaux au milieu du texte lors de la synthèse).*

Pour un livrable de type "PV de CSE" où chaque phrase compte, l'approche *Single-Prompt* est obsolète. Il faut repenser le backend `GptNewSummaryMaker` avec une approche dite **"Map-Reduce"** ou Agentique.

14.1 Comment font Claude, Cursor ou les outils avancés ?

Ils ne font **jamais** de traitement magique en un seul appel. Le secret de l'Agentic workflow réside dans l'utilisation du backend (Python, Java, etc.) comme **Chef d'Orchestre**. Le backend découpe le problème, fait des petits appels LLM ciblés, puis ré-assemble. Le code est fort pour le découpage et le contrôle (déterministe). L'IA est forte pour la rédaction (probabiliste).

14.2 Architecture Cible pour Agilotext (Pipeline Map-Reduce en Java)

Pour mettre cela en place côté Java (sans tout casser), l'évolution de `GptBaseNewChunkOneExecutor` se fera en 3 phases pour les jobs identifiés comme "massifs" :

```
flowchart TD
    TRANSCRIPT["Transcript Complet (3h, 50k tokens)"]

    subgraph "Phase 1 : MAP (Découpage & Rédation Parallèle)"
        CHUNK1["Chunk 1 (0-30 min)"]
        CHUNK2["Chunk 2 (30-60 min)"]
        CHUNK3["Chunk N..."]

        LLM1["Appel LLM 1<br>(Prompt : Rédige le PV détaillé de l'extrait)"]
        LLM2["Appel LLM 2<br>(Prompt : Rédige le PV détaillé de l'extrait)"]
        LLMN["Appel LLM N"]

        CHUNK1 --> LLM1
        CHUNK2 --> LLM2
        CHUNK3 --> LLMN
    end

    subgraph "Phase 2 : Traitement Local (Java)"
        MERGE["Concaténation brute des sous-PVs en mémoire"]
    end

    subgraph "Phase 3 : REDUCE (Formatage Final)"
        LLM_FINAL["Appel LLM Final<br>(Prompt : Applique le modèle HTML client [Bloc
2]<br>sur ce document pré-digéré)"]
    end

    LLM1 --> MERGE
    LLM2 --> MERGE
    LLMN --> MERGE

    MERGE --> LLM_FINAL
    LLM_FINAL --> OUTPUT["PV Complet, Fidéle, au format HTML"]
```

14.3 Règles d'implémentation pour Nicolas (Backend Java)

1. Le Splitter (Chunking Intelligent) : Plutôt que de découper sauvagement tous les 15 000 caractères, le backend doit découper intelligemment :

- Le plus propre : Vérifier les timestamps pour couper pendant un silence ou un changement de locuteur.
- Prévoir un "Overlapping" (chevauchement) de 500 mots d'un chunk à l'autre pour ne pas couper un débat au milieu d'une idée.

2. Le Prompt "Map" (Traitement des morceaux) :

- L'appel LLM de première ligne ne doit PAS inclure le template HTML lourd du client.
- *Instruction système :* "Tu es un greffier. Voici une portion de réunion. Extrais et rédige toutes les décisions, débats et positions des locuteurs de manière la plus exhaustive possible, sans formater. Ne fais aucune conclusion finale."

3. Le Prompt "Reduce" (Formatage final) :

- Une fois les morceaux de PV rassemblés (qui font maintenant peut-être 5 000 mots au lieu de 50 000), on fait l'appel terminal.
- *Instruction système :* Le Bloc 1 + Le Bloc 2 (Template client) + Le texte assemblé.
- L'appel final sert uniquement à appliquer le style, garantir que les tableaux du template sont remplis, et lisser le texte.

14.4 Pourquoi intégrer ça (ROI) ?

- **Fiabilité à 100% :** Un PV de 4 heures passera toujours, sans timeout Mistral.
- **Précision ultime :** Le modèle n'a "que" 30 minutes de texte à traiter par requête interne, il n'oublie donc aucune remarque d'un membre du CSE.
- **Parallélisation :** En Java, `CompletableFuture` ou l'ExecutorService permet d'envoyer les 8 requêtes Mistral (pour un découpage en 8) "en même temps". Le traitement d'un transcript de 3h ne sera pas plus long que celui de 30 minutes au niveau de l'attente API.

---

15. Critères d'Acceptation (Tests Formels)

#	Test	Résultat attendu
1	Audio seul, pas de document	**Identique** à aujourd'hui (golden path, zéro régression)
2	Audio + ODJ PDF avec 5 noms	≥ 4 noms dans Word Boost envoyé à Speechmatics
3	Audio + document + template HTML client **avec questions**	Structure HTML **respectée**, variables intactes, Bloc 3 visible mais séparé
4	`redoSummary` avec **autre** `promptId`	Nouveau template appliqué, **même** Bloc 3 contexte
5	`redoSummary` **même** `promptId` après suppression document	CR sans contexte (Bloc 3 vide)
6	PDF scanné (image)	Message d'erreur clair, transcription lancée **sans** boost

7	PDF de 50 pages	Analyse plafonnée (10 pages), pas de timeout
8	Enregistrement live + preAnalyze	Word Boost prêt au moment de l'upload WAV
9	Job terminé (succès ou erreur)	**Aucun** boost éphémère résiduel en BDD
10	Date "Comité du 12 mars 2026" dans le doc	Date proposée = 2026-03-12, pas date du jour
11	Mobile (api.ts) sans les nouveaux champs	API fonctionne normalement (rétrocompatibilité)
12	Upload .exe renommé en .pdf	Rejeté par magic bytes
13	Titre IA généré après transcript	displayTitle rempli, pas dans filename/clé stockage
14	Utilisateur renomme manuellement puis titre IA tente d'écraser	Titre user **conservé**, titleSource = "user"
15	Gros template HTML (3000 tokens) + document contexte + long transcript	Bloc 3 réduit (pas Bloc 2), pas de dépassement fenêtre
16	Profil onboarding "Dirigeant" + document sans participants	Le CR ne dit PAS "réunion dirigée par [nom du compte]"
17	Speaker enrollment laptop + même personne dictaphone	Doc Speechmatics recommande multi-clips — UX prévient l'utilisateur
18	Dossier supprimé contenant 3 jobs	Jobs reviennent à la racine, pas supprimés

---

16. Vague 8 — Mémoire de Dossier (inspirée Graphify)

> *[!NOTE]* ***Inspiration :*** *Graphify — outil qui construit un graphe de connaissances d'un projet une seule fois, puis le requête à chaque session IA au lieu de relire tous les fichiers. Concept transposé aux dossiers clients Agilotext. Voir analyse complète du transcript.*

14.1 Le problème résolu

```
AUJOURD'HUI (V1-V7) :
  Job 1 "Vente Dupont" : Audio + ODJ.pdf → analyse → CR
  Job 2 "Vente Dupont" : Audio            → repart de zéro !
  Job 3 "Vente Dupont" : Audio + ODJ2.pdf → aucun lien avec Job 1

  → Chaque réunion est une ÎLE ISOLÉE, sans mémoire des précédentes

AVEC V8 (Mémoire de Dossier) :
  Dossier "Vente Dupont-Martin" :
    ├── Job 1 : Audio + ODJ → NOURRIT le contexte du dossier
    ├── Job 2 : Audio seul → le dossier CONNAÎT déjà les noms
    ├── Job 3 : Audio → le CR peut citer les décisions de Job 1
    └── Contexte dossier : participants, décisions, termes, dates

  → Chaque réunion ENRICHIT la mémoire du dossier
```

14.2 Architecture technique

```
flowchart TB
    subgraph "Dossier 'Vente Dupont'"
        JOB1["Job 1<br>CR + contexte"] -->|"enrichit"| FOLDER_CTX["Contexte
Dossier<br>(JSON cumulatif)"]
        JOB2["Job 2<br>CR + contexte"] -->|"enrichit"| FOLDER_CTX
        JOB3["Job 3<br>nouvel audio"] -->|"lit"| FOLDER_CTX
        FOLDER_CTX -->|"Word Boost dossier"| JOB3
        FOLDER_CTX -->|"Bloc 3 enrichi"| JOB3
    end

    FOLDER_CTX --> FOLDER_BOOST["Word Boost persistant<br>du dossier"]
    FOLDER_CTX --> FOLDER_MEMORY["Résumé cumulatif<br>des réunions précédentes"]
```

14.3 Fonctionnement détaillé

```
CONTEXTE DOSSIER (FolderContext) :
  1. ACCUMULATION AUTOMATIQUE
     → À la fin de chaque job dans le dossier, extraire :
       - Participants confirmés du CR
       - Décisions/actions identifiées
       - Termes techniques récurrents
       - Dates clés mentionnées
     → Fusionner dans le FolderContext (JSON persistant)

  2. WORD BOOST PERSISTANT PAR DOSSIER
     → Tous les noms/termes du dossier = automatiquement
       dans le Word Boost des PROCHAINS jobs de ce dossier
     → Pas besoin de re-glisser le PDF à chaque réunion
     → Dédoublonnage avec le boost compte

  3. INJECTION CR AVEC MÉMOIRE
     → Bloc 3 enrichi : "Ceci est la 3ème réunion du dossier.
       Participants habituels : Jean Dupont (DG), ...
       Décisions précédentes :
         - Réunion 1 (12/04) : validation du budget Q1
         - Réunion 2 (19/04) : signature dans 15 jours"
     → Budget tokens : résumé plafonné (1500 tokens max)

  4. CYCLE DE VIE
     → Créé automatiquement quand un dossier a ≥ 2 jobs
     → Mis à jour après chaque nouveau CR dans le dossier
     → Supprimé si le dossier est supprimé
     → RGPD : même TTL que les jobs du dossier
```

14.4 Pourquoi c'est un game-changer

Aspect	Sans V8	Avec V8

Notaire, 10 réunions même dossier	Re-expliquer le contexte à chaque fois	Le système **se souvient** de tout
Noms propres	Re-glisser le PDF à chaque réunion	Déjà dans le **Word Boost du dossier**
Traçabilité	Chaque CR est isolé	Le CR cite les **décisions des réunions précédentes**
Lock-in produit	Facile de migrer (jobs isolés)	**Impossible** de migrer le graphe de connaissances cumulé
Valorisation	Outil de transcription	**Mémoire institutionnelle artificielle**

14.5 Prérequis

- ✅ V4 (Dossiers + `folderId`) doit être stable
- ✅ V2 (JobContext persisté par job) doit être stable
- ⚠️ **V1 de l'implémentation = JSON simple** (pas de graph database type Neo4j)
- ⚠️ Extraction post-CR (résumé structuré des décisions) = nouvel appel LLM léger

14.6 Ce que ce n'est PAS

- ❌ Ce n'est pas un vrai Knowledge Graph (Neo4j, ArangoDB) — trop complexe pour V1
- ❌ Ce n'est pas une recherche sémantique globale (RAG sur tous les CR) — V9+
- ❌ Ce n'est pas de la sync temps réel entre utilisateurs
- ✅ C'est un **JSON cumulatif par dossier**, enrichi après chaque job, injecté dans le Bloc 3 et le Word Boost des jobs suivants

---

17. Hors Périmètre V1 (explicitement)

- ❌ OCR serveur pour PDF scannés
- ❌ Multi-documents (un seul par job en V1)
- ❌ XLSX / tableurs
- ❌ Multimodal (vision page par page)
- ❌ Sous-dossiers (MVP = plat)
- ❌ MCP / mail (V7)
- ❌ Speaker ID (V6)
- ❌ Mémoire cumulative de dossier (V8)
- ❌ Traitement Map-Reduce Massif (V9 - Peut être avancé si la demande CSE est urgente)
- ❌ Garantie zéro faute (ASR + LLM = probabiliste)

---

18. Questions Ouvertes

> *[!IMPORTANT]* ***À trancher avant développement :***

Pour Florian (Product/UX)

1. **Preview des noms** : obligatoire dès V1 pour notaires/avocats (recommandé), ou acceptable en V2 ?
2. **Date réunion** : date picker natif HTML ou champ texte libre ? Prérempli à aujourd'hui ou vide ?
3. **Profil onboarding** : quel niveau de visibilité donner à l'utilisateur ? ("Nous utilisons votre profil pour améliorer..." vs injection silencieuse ?)

4. **Speaker enrollment** : UX "envoyer un lien à un collègue" vs "uploader un enregistrement" — priorité pour V6 ?
5. **Quotas dossiers/voix par offre** : les chiffres proposés (§11) sont-ils cohérents avec le pricing actuel ?
6. **Mémoire de dossier (V8)** : le client doit-il voir « Contexte de 3 réunions précédentes injecté » ou c'est transparent ?

Pour Nicolas (Backend)

7. **Apache Tika** : déjà dans le POM Maven ? Sinon quelle lib pour PDF/DOCX ?
8. **sendMultipleAudio** : ajout champ multipart optionnel — faisable sans casser ?
9. **job_word_boost_2** : réutilisable pour le boost éphémère par job, ou nouvelle table ?
10. **GptNewSummaryMaker** : où injecter le Bloc 3 dans le code actuel ? Avant ou après `text` dans le prompt ? Chunking compatible ?
11. **displayTitle** : nouveau champ dans le DTO `getJobsInfo` — estimation de la charge ?
12. **Speaker enrollment** : le connecteur Java vers Speechmatics supporte-t-il déjà `get_speakers: true` et `speaker_diarization_config.speakers` ?
13. **renameTranscriptFile** : plan de dépréciation ou cohabitation avec `PATCH /jobs/{id}` pour `displayTitle` ?
14. **Map-Reduce (V9)** : `GptBaseNewChunkOneExecutor` est-il facilement refactorisable en `GptMapReduceExecutor` avec lancement de requêtes parallèles via Java Threads ?

ANNEXE A — Index des Fichiers par Vague

Cette annexe garantit que **chaque fichier de l'infra actuelle** impacté par une vague est identifié. Référence à utiliser pour les tickets / sprints.

Vague 1 — Contexte amont (Upload doc + Word Boost éphémère)

Fichier	Action	Détail
ent.js	✏️ MODIFIER	Ajouter zone dropzone `contextFile` + validation MIME/taille dans le FormData avant `sendWithRetry`
pro.js	✏️ MODIFIER	Même modification que ent.js (quotas Pro)
free.js	✏️ MODIFIER	Même modification (quotas Free)
Backend `sendMultipleAudio` servlet	✏️ MODIFIER	Accepter champ optionnel `contextFile` (multipart)
Backend `DocumentAnalyzerService`	🆕 CRÉER	Extraction Tika + analyse LLM → JSON structuré
Backend `EphemeralWordBoostService`	🆕 CRÉER	Construction boost éphémère + merge + plafonds
Backend `preAnalyzeContextDocument`	🆕 CRÉER	Endpoint pour le cas live

Backend tables SQL `job_context`	🆕 CRÉER ou adapter `job_word_boost_2`	Stockage contexte analysé par job
specs/dashboard-prompt-backend-extensions.md	📖 RÉFÉRENCE	Endpoints contexte déjà spécifiés (setJobContext, addJobContextAttachment)

Vague 2 — CR Enrichi (Injection Bloc 3)

Fichier	Action	Détail
Backend `GptNewSummaryMaker`	✏️ MODIFIER	Insérer Bloc 3 (contexte) entre Bloc 2 (template) et Bloc 4 (transcript)
Backend `GptBaseNewChunkOneExecutor`	✏️ MODIFIER	Budget tokens global = Bloc1+Bloc2+Bloc3+Bloc4, compression Bloc 3 si nécessaire
relance-compte-rendu.js	📖 VÉRIFIER	S'assurer que le `redoSummary` avec `promptId` continue de fonctionner (Bloc 2 change, Bloc 3 stable)
Code-modeles-compte-rendu.js	📖 VÉRIFIER	Aucune modification nécessaire si le backend gère correctement l'injection
Dashboard (ent/pro/free.js)	✏️ MODIFIER	Ajouter champ date réunion (date picker)

Vague 3 — Titres Intelligents

Fichier	Action	Détail
Backend `getJobsInfo`	✏️ MODIFIER	Exposer `displayTitle`, `titleSource`, `folderId` dans le DTO
Backend `PATCH /jobs/{id}`	🆕 CRÉER	Nouveau endpoint pour title + folder
Backend `TitleGeneratorService`	🆕 CRÉER	LLM léger sur extrait transcript → titre
Code-changement-audio.js	✏️ MODIFIER	Afficher `displayTitle` dans le rail au lieu de `filename`
Mobile api.ts	✏️ MODIFIER	Mapper `displayTitle` dans `getJobsInfo` + helper `getJobDisplayTitle()`
Mobile JobCard.tsx	✏️ MODIFIER	Utiliser `getJobDisplayTitle(job)`
Mobile JobsListScreen.tsx	✏️ MODIFIER	Renommage → PATCH au lieu de `renameTranscriptFile`
CDC Titres	📖 RÉFÉRENCE	§3.7 séparation fichier/titre, §10 auto-critique
Plan 20/20 Titres	📖 RÉFÉRENCE	Phases A-E détaillées

Vague 4 — Dossiers

Fichier	Action	Détail
Backend CRUD `/folders`	🆕 CRÉER	GET/POST/PATCH/DELETE + `jobCount`
Backend `PATCH /jobs/{id}`	✏️ MODIFIER	Accepter `folderId`
Dashboard (ent/pro/free.js)	✏️ MODIFIER	UI dossiers dans le dashboard
Code-changement-audio.js	✏️ MODIFIER	Filtrage par dossier dans le rail
Mobile JobsListScreen.tsx	✏️ MODIFIER	Filtre/chips dossier + "Déplacer vers..."
Mobile `jobsListCache.ts`	✏️ MODIFIER	Rétrocompat champs optionnels

Vague 5 — Profil Utilisateur

Fichier	Action	Détail
Backend `UserProfileContextService`	🆕 CRÉER	Snapshot Memberstack + onboarding par job
Backend `GptNewSummaryMaker`	✏️ MODIFIER	Intégrer le profil dans le Bloc 3
Intégration Memberstack API	✏️ MODIFIER	Récupérer prénom, nom, plan, données onboarding

Vague 6 — Speaker Identification

Fichier	Action	Détail
Backend connecteur Speechmatics Java	✏️ MODIFIER	Supporter `get_speakers: true`, `speaker_diarization_config.speakers`
Backend `SpeakerEnrollmentService`	🆕 CRÉER	Enrollment, stockage identifiers, quota
Dashboard	✏️ MODIFIER	UI enregistrement voix + invitations
Mobile	✏️ MODIFIER	Écran d'enrollment intégré
mount-streaming.js	📖 VÉRIFIER	AudioWorklet compatible avec enrollment court

Vague 7 — MCP

Fichier	Action	Détail
agilotext-mcp-server/	✏️ MODIFIER	Ajouter source MCP → `JobContext`
Backend `JobContext`	✏️ MODIFIER	Nouvelle source typée `mcp_email`

Vague 8 — Mémoire de Dossier

Fichier	Action	Détail
Backend `FolderContextService`	🆕 CRÉER	Accumulation JSON par dossier, enrichissement post-CR
Backend `FolderWordBoostService`	🆕 CRÉER	Boost persistant par dossier
Backend `GptNewSummaryMaker`	✏️ MODIFIER	Bloc 3 enrichi avec résumé réunions précédentes
Backend tables SQL `folder_context`	🆕 CRÉER	JSON cumulatif par dossier
Backend post-CR hook	🆕 CRÉER	Extraction décisions/actions de chaque CR terminé

ANNEXE B — Fichiers Webflow/GitHub : Mapping Complet

Référence pour s'assurer qu'aucun script n'est oublié lors des déploiements. Source : [FEATURES_TRACKING.md]

Embed Webflow	Fichier GitHub	Impacté par
`code-css`	`Code-editor-css.js`	V1 (styles dropzone)
`code-rename-menu-css`	`Code-rename-menu-css.js`	V3 (rename → PATCH)
`code-css-chat`	`Code-chat-css.js`	—
`code-rail-css`	`Code-rail-css.js`	V3-V4 (displayTitle, dossiers)
`code-token-resolver`	`token-resolver.js`	—
`code-orchestrator`	`orchestrator.js`	V1 (ajout contextId dans credentials)
`code-ready-count`	`ready-count.js`	—
`code-lecteur-audio`	`Code-lecteur-audio.js`	—
`code-main-editor`	`Code-main-editor.js`	V2 (affichage contexte dans métadonnées job)
`code-changement-audio`	`Code-changement-audio.js`	V3-V4 (displayTitle, filtrage dossier)
`code-chat`	`Code-chat.js`	V8 (chat IA cross-réunion dans un dossier)
`code-save_transcript`	`Code-save_transcript.js`	—
`code-relance-compte-rendu`	`relance-compte-rendu.js`	V2 (vérifier redo + contexte stable)
`code-editor-main`	`editor-main.js`	V1 (charger nouveau script contexte)

Dashboard	`ent.js` / `pro.js` / `free.js`	V1 (dropzone) V2 (date) V4 (dossiers)
Streaming	`mount-streaming.js`	V1 (preAnalyze) V6 (speaker)

ANNEXE C — Documents Mobile à Synchroniser

Document	Rôle
CDC Mobile iOS/Android	Roadmap mobile next
CDC Mobile Parité Webapp	Features à aligner web/mobile
Audit Sécurité Mobile	Contraintes sécurité
Audit UX Mobile	UX à respecter
Plan Mobile Extension V2	Extension features mobile